ELEC 475: Lab 3 — Post-Lab

Pet Nose Localization Correction

Tyler Wilson and Michael Greenspan

Oct. 22, 2026

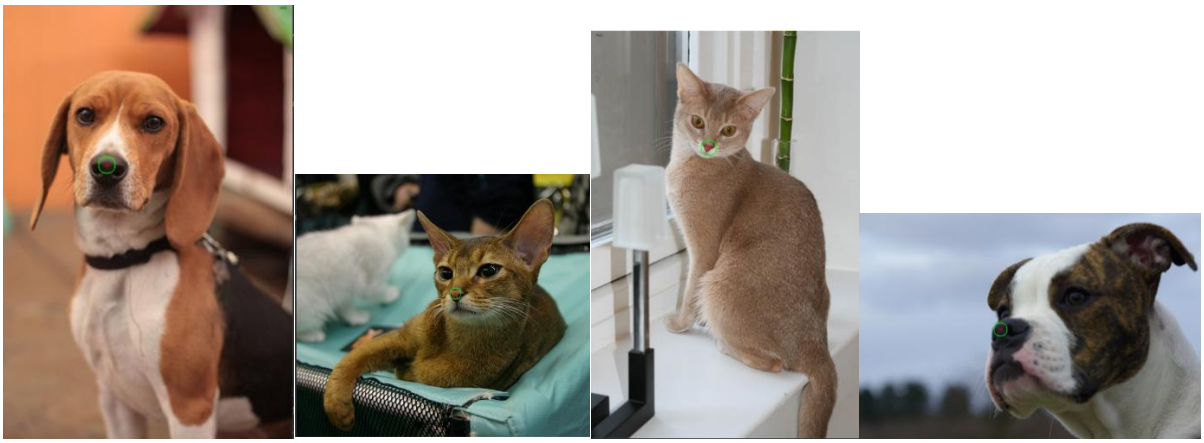


Table of Contents

Step 1: Correction Dataset 2
Step 2: Correction Model 3
Step 3: Training 4
Step 4: Evaluating the Correction 4
Step 5: Modified Correction Model, Training and Evaluation 5
Step 6: Report 6
Step 7: Submission Format 7

Step 1: Correction Dataset

In the In-Lab, each model predicted the nose position from the whole image. In the Post-Lab, you will build a correction model that refines a prediction. It looks only at a small patch of the image centered on the first prediction, and predicts the offset from the patch center to the true nose:

$$(\hat{u}', \hat{v}') = (c_u, c_v) + 64 \cdot (t_u, t_v)$$

where (c_u, c_v) is the patch center (the first prediction, rounded to the nearest pixel), (t_u, t_v) is the model's output, and $(\hat{u}', \hat{v}')$ is the corrected prediction. Each patch covers a 128 × 128 pixel region of the original image, resized to 64 × 64, so the offset is divided by 64 (`HALF_PATCH`) to keep the model's targets roughly between −1 and 1.

The correction model is trained only on the training set. Since a training image has no first prediction (i.e. ground truth), you will create one, in the form of a patch center placed at a random offset from the true nose.

Complete the following in `utils/correction_dataset.py`:

Function	Requirements
`extract_patch(img, center)`	Crop the 128 × 128 region of `img` centered on `center` with `img.crop()`, and resize it to 64 × 64 with `Image.BILINEAR`. Any part of the region outside the image is black.
`CorrectionDataset.__getitem__(idx)`	Return one training patch and its target, as described below.

`CorrectionDataset` is a subclass of your Pre-Lab `PetNoseDataset`. It reuses the parsed training labels in `self.samples`, a list of (name, u, v) tuples, and the image folder in `self.img_dir`. If your `PetNoseDataset` stores these under different names, rename them.

For `__getitem__`:

1. Open the training image and get its true nose position (u, v).
2. Choose a random patch center: add a random offset, uniform between −40 and 40 pixels (`MAX_OFFSET`), to u and to v, and round to the nearest pixel.
3. Extract the patch centered on this point with `extract_patch`. The target is the offset from the patch center to the true nose, divided by 64.
4. If `self.augment` is `True`, apply two augmentations:
    a. With probability 0.5, flip the patch horizontally, and update the target to match.
    b. Apply colour jitter (`self.color_jitter`), and update the target if needed.
5. Return the normalized patch tensor and the target.

Run the reality check, which saves eight patches without augmentation and eight with:

```
python -m utils.correction_dataset
```

Open `plots/correction_patches.png` and `plots/correction_patches_aug.png`. In every patch, the green circle (the target) should sit on the nose. If the circles are misplaced only in some augmented patches, check how your flip updates the target.

Step 2: Correction Model

Implement `CorrectionNet` in `models/correction.py`. It has four convolution blocks and two fully connected layers. Each block applies a 3 × 3 convolution with `padding=1`, batch normalization, a ReLU, then 2 × 2 max pooling with stride 2:

Layer	Definition	Output Shape
Input		(3, 64, 64)
Block 1	`nn.Conv2d(3, 32, 3, padding=1)`, `nn.BatchNorm2d(32)`	(32, 32, 32)
Block 2	`nn.Conv2d(32, 64, 3, padding=1)`, `nn.BatchNorm2d(64)`	(64, 16, 16)
Block 3	`nn.Conv2d(64, 128, 3, padding=1)`, `nn.BatchNorm2d(128)`	(128, 8, 8)
Block 4	`nn.Conv2d(128, 128, 3, padding=1)`, `nn.BatchNorm2d(128)`	(128, 4, 4)
Flatten		2,048
`fc1` → ReLU	`nn.Linear(2048, 256)`	256
`fc2`	`nn.Linear(256, 2)`	2

Verify your model:

```
python -m models.correction
```

The summary should report 766,594 total parameters.

Step 3: Training

`train_correction.py` is complete. It trains `CorrectionNet` on `CorrectionDataset` for 20 epochs with the Adam optimizer (learning rate 10^{-3}, cosine schedule), a batch size of 64 and an MSE loss. Train two versions, one without and one with augmentation:

```
python train_correction.py --no-augment --save-model
   checkpoints/correction_noaug.pth --plot-file
   plots/loss_correction_noaug.png

python train_correction.py --save-model checkpoints/correction_aug.pth
   --plot-file plots/loss_correction_aug.png
```

where the arguments refer to the following:

- `--no-augment`: disable the flip and colour jitter augmentations
- `--save-model`: where the trained weights are saved
- `--plot-file`: where the loss curve is saved

Each epoch prints the training loss and the mean offset error in original-image pixels. Review the loss curves to confirm that both models trained properly.

Step 4: Evaluating the Correction

`correct.py` is complete. It corrects the In-Lab test-set predictions of SnoutNet, SnoutNet-V and the mean ensemble: for each test image, it extracts a patch centered on the first prediction, runs your correction model, and saves the corrected predictions to `predictions/`. Run it for both versions of the correction model:

```
python correct.py --checkpoint checkpoints/correction_noaug.pth
python correct.py --checkpoint checkpoints/correction_aug.pth
```

Then run `evaluate.py` again:

```
python evaluate.py
```

`evaluate.py` adds the corrected predictions to its results automatically. The accuracy and cost tables and `plots/ced.png` now include six more rows: SnoutNet, SnoutNet-V and the mean ensemble, each corrected with and without augmentation. The cost of a corrected model includes the first model(s) and the correction model. `evaluate.py` also saves the best and worst corrected predictions, for example `plots/best_worst_snoutnet_correction_aug.png`.

The correction model never sees the test set during training, and nothing in this step is adjusted using test results.

Step 5: Modified Correction Model, Training and Evaluation

Modify the architecture of the correction model of Step 2, with the aim of improving performance. Implement it as `CorrectionNetModified` in `models/correction.py`, leaving `CorrectionNet` unchanged. It takes the same input and returns the same output as `CorrectionNet`. Verify both models:

```
python -m models.correction
```

Then repeat Steps 3 and 4 with the `--modified` flag, which selects `CorrectionNetModified`:

```
python train_correction.py --modified --no-augment --save-model
   checkpoints/correction_mod_noaug.pth --plot-file
   plots/loss_correction_mod_noaug.png
python train_correction.py --modified --save-model
   checkpoints/correction_mod_aug.pth --plot-file
   plots/loss_correction_mod_aug.png
python correct.py --modified --checkpoint
   checkpoints/correction_mod_noaug.pth
python correct.py --modified --checkpoint
   checkpoints/correction_mod_aug.pth
python evaluate.py
```

Use these checkpoint names exactly: `evaluate.py` only includes corrected predictions from checkpoints named `correction_noaug`, `correction_aug`, `correction_mod_noaug` and `correction_mod_aug`.

Step 6: Report

The report should include a title, your name(s) and student number(s). If working in a team of two, only submit once in OnQ, but record both partners' information. Include the following sections in the report.

1. **Model:** This section should contain all details of CorrectionNet and CorrectionNetModified, sufficient to recreate both models. This could be in textual form, diagram form, etc.
2. **Training:** This section should include all details of the correction models' training, sufficient to exactly reproduce it. This should include the training data and how patches are sampled, the augmentations, the optimization method, optimization hyperparameters, learning rate scheduling, and all other relevant details.
3. **Outputs:** Include the following outputs:
    a. The accuracy and cost tables from `evaluate.py`, including the In-Lab rows and the corrected SnoutNet, SnoutNet-V and mean ensemble, for both correction models, with and without augmentation.
    b. The cumulative error distribution plot (`plots/ced.png`), including the In-Lab results and the corrected SnoutNet, SnoutNet-V and mean ensemble, for both correction models, with and without augmentation.
    c. The best and worst predictions of SnoutNet, corrected SnoutNet, and corrected SnoutNet with augmentation (best_worst_snoutnet.png, best_worst_snoutnet_correction_noaug.png and best_worst_snoutnet_correction_aug.png).
    d. The loss curves of all four correction models.
4. **Analysis:** This section should analyze the results of the In-Lab and Post-Lab.
    a. **Failure cases:** Look at each model's worst predictions. What do the hardest images have in common, and why might these be difficult for a nose localizer?
    b. **Pretraining:** SnoutNet-A and SnoutNet-V start from ImageNet weights, while SnoutNet is trained from scratch. Why does pretraining help so much for this task?
    c. **Ensembles:** Why does combining the three models help or not help here? When would you expect an ensemble to outperform its best member?
    d. **Correction model:** The correction model only sees a small patch around the first prediction. Does it help refine predictions, and when can it not help? Compare the improvement for SnoutNet, SnoutNet-V and the mean ensemble.
    e. **Augmentation:** How did flipping and colour jitter affect the correction model, and why?
    f. **Modified correction model:** How did you change the architecture of the correction model? Did it improve performance? Why, or why not?

Step 7: Submission Format

The report and the complete working source directory should be compressed into a single <zzz>.zip file, where filename <zzz> is replaced with your student number. If you are in a team of two, then concatenate both student numbers, separated by an underscore.

All code should be executable within a terminal. Include a text file with the appropriate terminal lines. The zipped directory should not contain the dataset, the provided weights (`snoutnet.pth`, `alexnet.pth` and `vgg16.pth`) or the `predictions/` folder. The zipped directory must include the following:

- All Pre-Lab, In-Lab and Post-Lab code
- `checkpoints/correction_noaug.pth`, `checkpoints/correction_aug.pth`, `checkpoints/correction_mod_noaug.pth` and `checkpoints/correction_mod_aug.pth`
- All plots in `plots/`
- The Post-Lab report

The lab is worth a total of 7 marks. The marking rubric is as follows:

Item	Marks
In-Lab	3 (marked in lab)
Step 1: Correction dataset and augmentation	1
Steps 2–4: Correction model, training and evaluation	1
Step 5: Modified correction model, training and evaluation	1
Step 6, 7: Post-Lab report, and correct submission format	1
Total	**7**

The Post-Lab is due (without late penalty) November 5th, 2026 before 23:59 ET.